UNIT-5

12/09/14.

GRAPH THEORY.

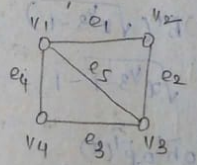
Graph theory plays an important role in computer science applications such as logic design, switching theory, computer graphics and operating systems etc....

Graph:

A graph $G(V, E)$ consists of a non-empty set of vertices (V) and edges (E) may or may not be empty of a graph.

We denote a graph as $G(V, E)$ or simply G

Eg:



Where

$V(G) = \{v_1, v_2, v_3, v_4\}$

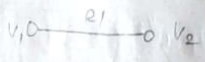
$E(G) = \{e_1, e_2, e_3, e_4, e_5\}$

Note:

We can represent no of different diagrams (graphs) for same number of vertices.

* Any pair of vertices which are connected by an edge is called as adjacent vertices.

Eg:

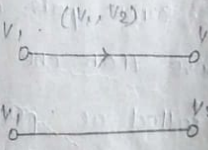


where

v_1 & v_2 are adjacent vertices.

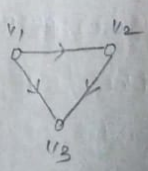
* In a graph $G(V, E)$, an edge is associated with an ordered pair of vertices is called as directed edge and while an edge is associated with an unordered pair of vertices is called as undirected edge

Eg:



* A graph in which every edge is directed, then it is called as directed graph (or) Digraph.

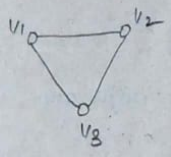
Eg:



Scanned by CamScanner

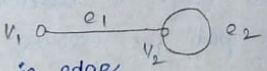
* A graph in which every edge is undirected, then it is called as an undirected graph.

Eg:



* An edge of graph which joins vertex to itself is called as loop.

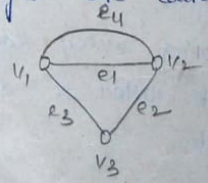
Eg:



e_1 is edge
Where e_2 is loop.

* In a graph, if some pair of vertices are joined by more than one edge, such ~~are~~ edges are called as parallel edges.

Eg:



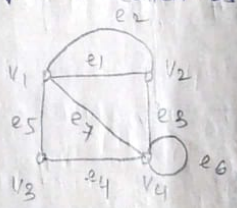
Here e_1 & e_4 are parallel edges.

Types of Graphs.

1. Multi-graph:

Any graph which contains parallel and loop edges is called as Multi-graph.

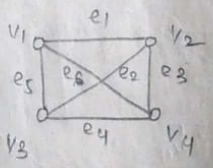
Eg:



2. Simple graph:

A graph with no parallel and loop edges is called as simple graph.

Eg:



3. Trivial graph:

A graph with single vertex having no edges is called as trivial graph.

Eg:

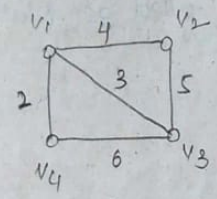


Scanned by CamScanner

4. Weighted Graph:

A graph which contains edges with weight is called as weighted graph.

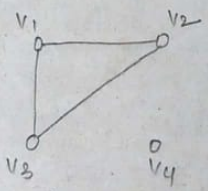
Eg:



Isolated Vertex:

In a graph, all vertices are connected except one vertex. that vertex is called as isolated vertex.

Eg:



Here v_4 is isolated vertex.

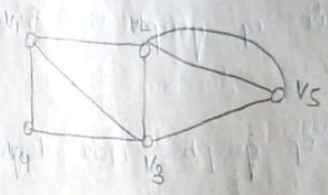
13/09/14.

Degree of graph:

let 'v' be a vertex in a graph 'G' then the degree of vertex 'v' in 'G' is no of edges incident with vertex 'v'.

It can be denoted as deg(v).

Undirected Graph:



deg(v_1) = 3

deg(v_2) = 4

deg(v_3) = 4

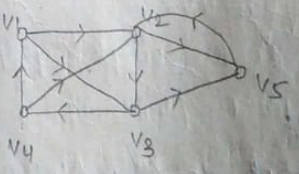
deg(v_4) = 2

deg(v_5) = 3

Degree of Graph = 4

Highest degree of vertex (graph) is known as degree of graph.

Directed Graph:



deg(v_1) = Indegree + outdegree

= 1 + 2 = 3.

deg(v_2) = 3 + 2 = 5

deg(v_3) = 2 + 2 = 4

deg(v_4) = 1 + 2 = 3

Scanned by CamScanner

$\deg(v_5) = 2+1 = 3$

Degree of graph is 5.

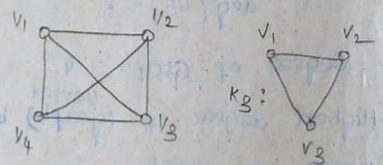
Note:

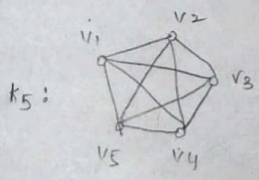
Degree of graph = Max degree of vertex.

Complete Graph:

A simple graph in which every pair of distinct vertices are adjacent is called as complete graph (K_n)

Eg:

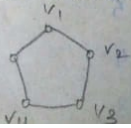


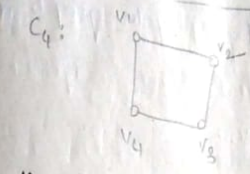


Cycle Graph:

A cycle graph of order 'n' is a connected graph, whose edges form a cycle of length 'n' and denoted by 'C_n'.

Eg:

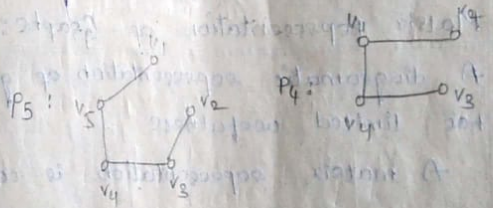




Path Graph:

A path graph of order 'n' is obtained by removing an edge from C_n graph; denoted by 'P_n'.

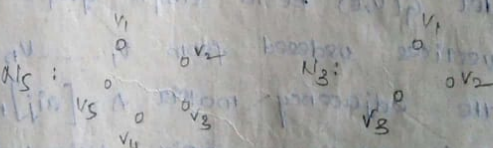
Eg:



Null Graph:

A null graph of order 'n' is a graph with n vertices and no edges denoted by N_n.

Eg:



Wheel Graph:

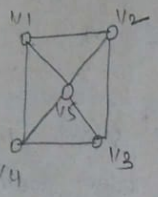
A wheel graph of order 'n' is obtained by joining a new vertex called "hub"

Scanned by CamScanner

to each vertex of a cycle graph denoted by W_n.

Eg:

W_5:



16/09/14.

Representation of Graphs:

Matrix Representation of Graphs:

A diagramatic representation of graph has limited usefulness.

A matrix representation is convenient and useful for more computer applications. For this purpose we have number of methods. Those are

(i) Adjacency matrix:

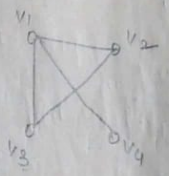
Let $G(V,E)$ be a simple graph with n vertices ordered from $V_1, \ldots, V_n$, then the adjacency matrix $A = [a_{ij}]_{n \times n}$ of G is an $n \times n$ matrix defined by elements.

$$a_{ij} = \begin{cases} 1 & \text{when } v_i \text{ is adjacent to } v_j \\ 0 & \text{otherwise.} \end{cases}$$

It can be denoted $A(G)$ or A_G.

Eg:

A graph G and find adjacency matrix?



Sol:

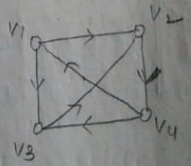
$n = 4$

$$A_G = \begin{array}{c} \\ V_1 \\ V_2 \\ V_3 \\ V_4 \end{array} \begin{array}{c} \begin{array}{cccc} V_1 & V_2 & V_3 & V_4 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} \end{array} \quad 4 \times 4$$

In case of directed graph.

$$a_{ij} = \begin{cases} 1 & \text{if } (v_i, v_j) \in E \\ 0 & \text{otherwise.} \end{cases}$$

Eg: Find adjacency matrix for following digraph.



Sol:

$n = 4$

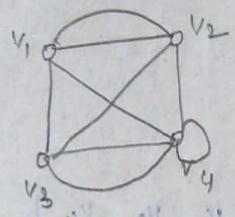
$$A_G = \begin{array}{c} \\ V_1 \\ V_2 \\ V_3 \\ V_4 \end{array} \begin{array}{c} \begin{array}{cccc} V_1 & V_2 & V_3 & V_4 \end{array} \\ \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{array} \quad 4 \times 4$$

Scanned by CamScanner

In case of Multigraph:

Eg: A graph G and find adjacency matrix.?

Sol.

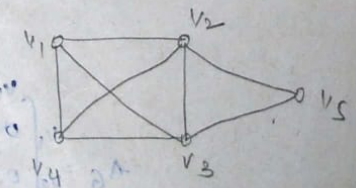


Sol. n = 4

	V_1	V_2	V_3	V_4
V_1	0	2	1	1
V_2	2	0	1	1
V_3	1	1	0	2
V_4	1	1	2	2

4x4

Eg: Find adjacency matrix for following graph.

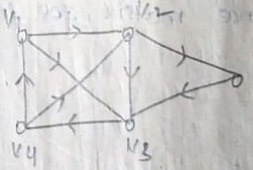


Sol. n = 5

	V_1	V_2	V_3	V_4	V_5
V_1	0	1	1	1	0
V_2	1	0	1	1	1
V_3	1	1	0	1	1
V_4	1	1	1	0	0
V_5	0	1	1	0	0

5x5

In case of directed graph.



	V_1	V_2	V_3	V_4	V_5
V_1	0	1	1	0	0
V_2	0	0	1	0	1
V_3	0	0	0	1	0
V_4	1	1	0	0	0
V_5	0	0	1	0	0

5x5

Scanned by CamScanner

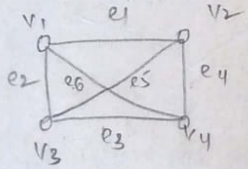
(iii) Incidence Matrix:

let G be a graph with n vertices and $V = \{v_1, v_2, \ldots v_n\}$ and $E = \{e_1, e_2, \ldots e_m\}$ define $n \times m$ matrix $I_G = [m_{ij}]_{n \times m}$. where

$$m_{ij} = \begin{cases} 1 & \text{when } v_i \text{ is incident with } e_j \\ 0 & \text{otherwise} \end{cases}$$

Eg:

1. Find incidence matrix for following graph.



Sol.

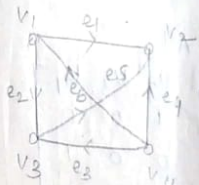
$$I_G = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{array} \begin{array}{c} \begin{array}{cccccc} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \end{array} \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix} \end{array}$$

4 × 6

In case of directed graph.

$$m_{ij} = \begin{cases} 1 & \text{if } v_i \text{ is the initial vertex of } e_j \\ -1 & \text{if } v_i \text{ is final vertex of } e_j \\ 0 & \text{otherwise.} \end{cases}$$

Eg:



Sol.

$$I_G = \begin{array}{c} \\ v_1 \\ v_2 \\ v_3 \\ v_4 \end{array} \begin{array}{c} \begin{array}{cccccc} e_1 & e_2 & e_3 & e_4 & e_5 & e_6 \end{array} \\ \begin{bmatrix} 1 & 1 & 0 & 0 & 0 & -1 \\ -1 & 0 & 0 & -1 & -1 & 0 \\ 0 & -1 & -1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 1 \end{bmatrix} \end{array}$$

Path Matrix:

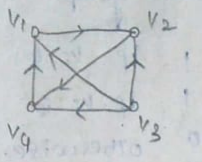
let G be a simple digraph having no parallel directed edges and $V = \{v_1, v_2, \ldots v_n\}$ be its vertex set. An $n \times n$ matrix $P = [P_{ij}]_{n \times n}$ is given by

$$P_{ij} = \begin{cases} 1 & \text{if there is a path from } v_i \text{ to } v_j \\ 0 & \text{otherwise} \end{cases}$$

$P = A + A^2 + A^3 + \ldots + A^n$ where n is no. of vertices.

Scanned by CamScanner

Eg: Find Path Matrix.



Sol. $n = 4$

$P = A + A^2 + A^3 + A^4$

$$A = A_G = \begin{array}{c} \\ V_1 \\ V_2 \\ V_3 \\ V_4 \end{array} \begin{array}{c} \begin{array}{cccc} V_1 & V_2 & V_3 & V_4 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} \end{array}$$

$$A^2 = A \cdot A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$A^3 = \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A^4 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

$P = A + A^2 + A^3 + A^4$

$$P = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \end{bmatrix} +$$

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 0 \end{bmatrix}$$

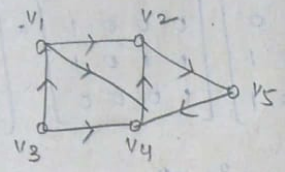
$$P = \begin{bmatrix} 1 & 2 & 0 & 1 \\ 1 & 1 & 0 & 2 \\ 4 & 4 & 0 & 4 \\ 2 & 1 & 0 & 1 \end{bmatrix}$$

In final result, if any value > 1, replace by 1 otherwise 0.

$$\therefore P = \begin{bmatrix} 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 0 & 1 \end{bmatrix}$$

Scanned by CamScanner

2. Find Path Matrix.



Sol.

$n = 5$

$P = A + A^2 + A^3 + A^4 + A^5$

$$A = A_G = \begin{array}{c} \\ V_1 \\ V_2 \\ V_3 \\ V_4 \\ V_5 \end{array} \begin{array}{c} \begin{array}{ccccc} V_1 & V_2 & V_3 & V_4 & V_5 \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \end{array}$$

$$A^2 = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$A^3 = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A^4 = \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

$$A^5 = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

$$P = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} +$$

$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 \end{bmatrix}$$

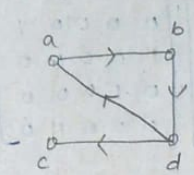
$$P = \begin{bmatrix} 0 & 4 & 0 & 3 & 3 \\ 0 & 1 & 0 & 2 & 2 \\ 1 & 3 & 0 & 4 & 2 \\ 0 & 2 & 0 & 1 & 2 \\ 0 & 2 & 0 & 2 & 1 \end{bmatrix}$$

$$P = \begin{bmatrix} 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \\ 0 & 1 & 0 & 1 & 1 \end{bmatrix}$$

Scanned by CamScanner

18/09/14.

1. Find Path matrix for following graph.



Sol. $n = 4$

$$P = A + A^2 + A^3 + A^4$$

$$A = A_G = \begin{array}{c} \\ a \\ b \\ c \\ d \end{array} \begin{array}{c} \begin{array}{cccc} a & b & c & d \end{array} \\ \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{array}$$

$$A^2 = A \cdot A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix}$$

$$A^3 = A^2 \times A$$

$$= \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$A^4 = A^3 \times A$$

$$= \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

$$P = A + A^2 + A^3 + A^4$$

$$= \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} + \begin{bmatrix} 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} + \begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} +$$

$$\begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

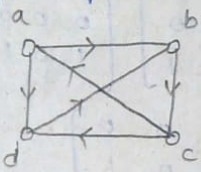
$$= \begin{bmatrix} 1 & 2 & 1 & 1 \\ 1 & 1 & 1 & 2 \\ 0 & 0 & 0 & 0 \\ 2 & 1 & 2 & 1 \end{bmatrix}$$

Scanned by CamScanner

$$\therefore P = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Warshall's Algorithm:

* Represent the following graph in matrix form by using Warshall's algorithm.



Sol: Matrix of relation (M_R) =

	a	b	c	d
a	0	1	0	1
b	0	0	1	0
c	1	0	0	1
d	0	1	0	0

$\therefore M_P =$

	a	b	c	d
a	0	1	0	1
b	0	0	1	0
c	1	0	0	1
d	0	1	0	0

$= P[0]$

c-a | a-b, a-d
c-b
c-d

$P[1] =$

	a	b	c	d
a	0	1	0	1
b	0	0	1	0
c	1	1	0	1
d	0	1	0	0

a-b, c-b, d-b | b-c
a-c
c-c
d-c

$P[2] =$

	a	b	c	d
a	0	1	1	1
b	0	0	1	0
c	1	1	1	1
d	0	1	1	0

a-c, b-c, c-c, d-c | c-a, c-b, c-c, c-d

a-a b-a c-a d-a
a-b b-b c-b d-b
a-c b-c c-c d-c
a-d b-d c-d d-d

$P[3] =$

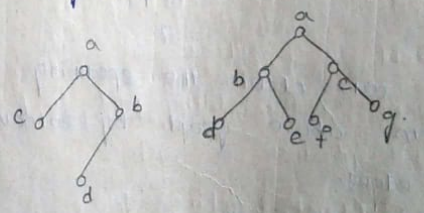
	a	b	c	d
a	1	1	1	1
b	1	1	1	1
c	1	1	1	1
d	1	1	1	1

$= P[4]$

* Spanning Trees:

A tree is a graph G which is connected and no cycles.

Eg:



Definition:

let G be a connected graph. A subgraph T of G is called as a spanning tree of G if it satisfies the following conditions.

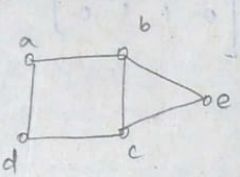
Scanned by CamScanner

(ii) T is tree
(iii) T contains all vertices of graph.

19/09/14

Example :

1.



Sol: we need to find out circuit rank of given graph

Circuit rank = $m-(n-1)$

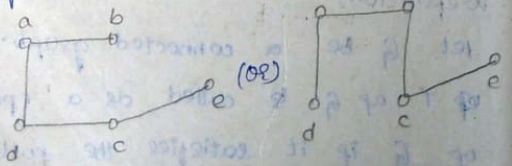
where
m = no of edges
n = no of vertices

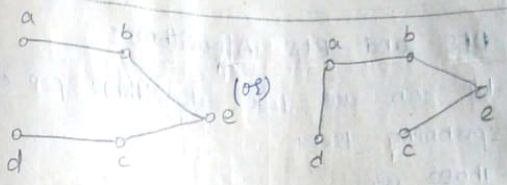
$m = 6$
$n = 5$

$= 6-(5-1)$
$= 6-4$
$= 2$.

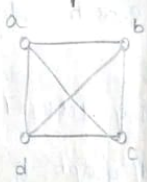
we can obtain the spanning tree from above graph by removing 2 edges.

Spanning tree is





2. Obtain spanning tree from given graph.



Sol: we need to find out circuit rank of given graph.

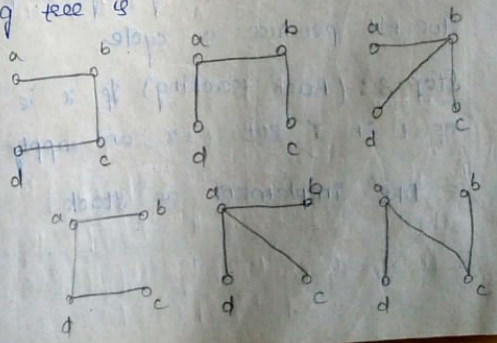
circuit rank = $m-(n-1)$

$m = 6$, $n = 4$

$= 6-(4-1)$
$= 6-3$
$= 3$

we can obtain the spanning tree from above graph by removing 3 edges

Spanning tree is



Scanned by CamScanner

20/09/14

DFS and BFS Algorithms:

We can use two algorithms for obtaining spanning tree.

Those are

1. Depth First Search algorithm.
2. Breadth First Search algorithm.

** 1. DFS Algorithm:

DFS for a spanning tree T

Input: A connected graph G with vertices $(v_1, v_2, \ldots v_n)$.

Output: A spanning tree T.

Step-1: (Visit a vertex). let v_1 be the root of T and set $L = v_1$.

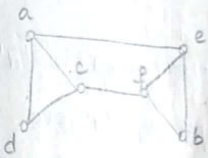
Step-2: (Find an unexamined edge and unvisited vertex to L) For all vertices adjacent to L, choose the edge and also confirm that adding edge to T that doesn't produce a cycle.

Step-3: (Back tracking) If x is the parent of L in T set $L = x$ and apply step-2

DFS implement as stack

Eg:

1. Implement Spanning Tree of following graph using DFS.



Sol. The all vertices in above graph in an order

a b c d e f

Now we select vertex 'a' as root node of T

Find the adjacent edges of vertex 'a'.

These are {a,c} {a,d} {a,e}.

Now we select {a,c} edge based on order c is next working vertex.

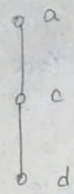


All adjacent edges of vertex 'c'

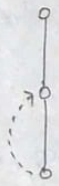
Those are {c,a} {c,d} {c,f}

Now we select {c,d} edge based on order that doesn't produce cycle.

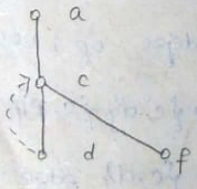
Scanned by CamScanner



Next working vertex (node) is 'd'.
The adjacent edges of 'd' are
{d,a} {d,c}
no more edge is selected. Now move to parent node of 'd'.



Next working node is 'c'
The adjacent edges of vertex 'c' are
{c,a} {c,d} {c,f}.
Now, we select {c,f} edge based on order and that doesn't produce cycle.

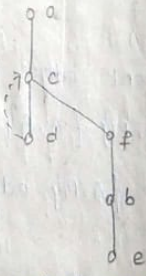


Next working node is 'f'
The adjacent edges of 'f' are
{f,b} {f,c} {f,e}

we select {f,b} edge based on order



Next working node is 'b'
All adjacent edges of vertex 'b' are
{b,e} {b,f}
Now we select {b,e} edge based on order and that doesn't produce cycle.



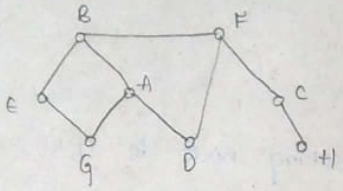
This is spanning tree of above graph by using DFS.

Result Order:

a c d f b e

Scanned by CamScanner

2. Implement spanning tree of following graph using DFS.



Sol. The all vertices in above graph in an order

A B C D E F G H.

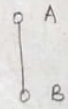
Now we select vertex 'A' as root node of T



Find the adjacent edges of vertex 'A'
Those are {A, B} {A, G} {A, D}

Now we select {A, B} edge based on order

B is next working vertex.



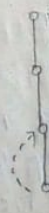
All adjacent edges of vertex 'B'
Those are {B, E} {B, F} {B, A}
Now we select {B, E} edge based on order. that doesn't produce cycle



E is next working node
All adjacent edges of vertex 'E'.
Those are {E, B} {E, G}
Now we select {E, G} edge based on order that doesn't produce cycle.



G is next working node.
All adjacent ~~vertices~~ edges of vertex 'G'
Those are {G, E} {G, A}
No more edge is selected.
Now move to parent node of 'G'



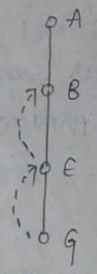
Scanned by CamScanner

E is next working node.

The adjacent edges of 'e' are

{E, B} {E, G}

No more edge is selected. Now move to parent node of 'E'

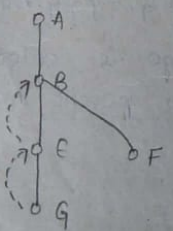


B is next working node.

The adjacent edges of 'B' are

{B, A} {B, E} {B, F}

Now we select {B, F} edge based on order that doesn't produce cycle.

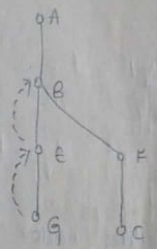


F is next working node.

The adjacent edges of 'F' are

{F, B} {F, D} {F, C}

Now we select {F, C} edge based on order that doesn't produce cycle.

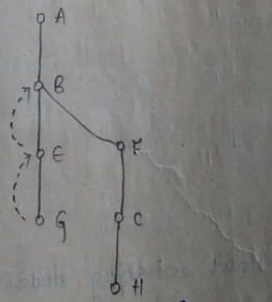


C is next working node.

The adjacent edges of 'C' are

{C, F} {C, H}

Now we select {C, H} edge based on order that doesn't produce cycle.



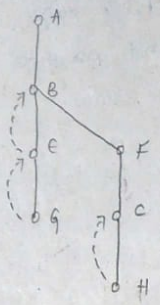
H is next working node.

The adjacent edges of 'H' are

{H, C}

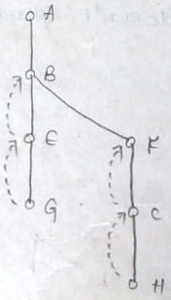
No more edge is selected. Now move to parent node of 'H'

Scanned by CamScanner

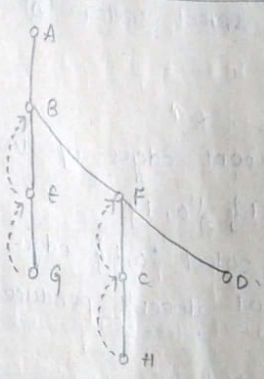


C is next working node
The adjacent edges of 'c' are
{C,F} {C,H}

No more edge is selected. Now move to parent node of 'c'



F is next working node
The adjacent edges of 'F' are
{F,B} {F,D} {F,C}
Now we select {F,D} edge based on order that doesn't produce cycle.



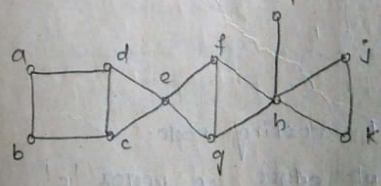
This is spanning tree of above graph by using DFS.

Result Order:

A B E G F C H D

12-09-14.

1. Derive the depth first spanning tree for following graph:



Ans. The all vertices in above graph are in an order

a b c d e f g h i j k

Scanned by CamScanner

Now we select vertex 'a' as root node of T



All adjacent edges of vertex 'a'

{a,b} {a,d}

Now we select {a,b} edge based on order that doesn't produce cycle.



b is next working node.

All adjacent edges of vertex 'b'

{b,a} {b,c}

Now we select {b,c} edge based on order that doesn't produce cycle.



c is next working node.

All adjacent edges of vertex 'c'

{c,b} {c,d} {c,e}

Now we select {c,d} edge based on order that doesn't produce cycle.

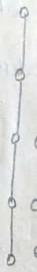


d is next working node.

All adjacent edges of vertex 'd'

{d,a} {d,c} {d,e}

Now we select {d,e} edge based on order that doesn't produce cycle.



e is next working node.

All adjacent edges of vertex 'e'

{e,a} {e,d} {e,f} {e,g}

Now we select {e,f} edge based on order that doesn't produce cycle.

Scanned by CamScanner

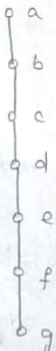


f is next working node.

All adjacent edges of vertex 'f'

{f,e} {f,g} {f,h}

Now we select {f,g} edge based on order that doesn't produce cycle.



g is next working node.

All adjacent edges of vertex 'g'

{g,e} {g,f} {g,h}

Now we select {g,h} edge based on order that doesn't produce cycle.

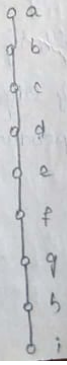


h is next working node.

All adjacent edges of vertex 'h'

{h,f} {h,g} {h,i} {h,j} {h,k}

Now we select {h,i} edge based on order that doesn't produce cycle.



Scanned by CamScanner

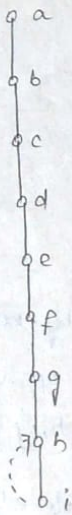
i is next working node.

All adjacent edges of vertex 'i'.

{i, h}

No more edge is selected.

Now move to parent node of 'i'.



h is next working node.

The adjacent edges of vertex 'h'

{h, f} {h, g} {h, i} {h, j} {h, k}

Now we select {h, j} edge based on order that doesn't produce cycle.

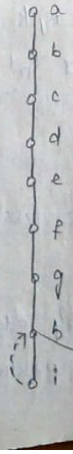


j is next working node.

The adjacent edges of vertex 'j'

{j, h} {j, k}

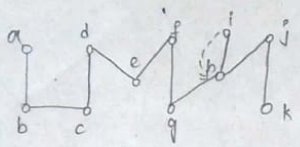
Now we select {j, k} edge based on order that doesn't produce cycle.



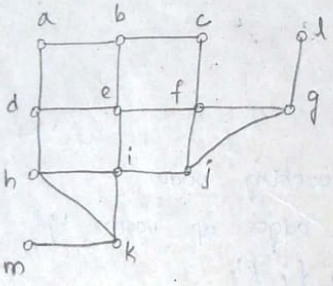
Scanned by CamScanner

Result order:

a b c d e f g h i j k



2.



Sol. The all vertices in the above graph in an order

a b c d e f g h i j k l m.

Now we select vertex 'a' as root node of T

The adjacent edges of vertex 'a'

{a,b} {a,d}

Now we select {a,b} edge based on order that doesn't produce cycle.

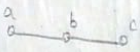


b is next working node.

The adjacent edges of vertex 'b'

{b,a} {b,c} {b,e}

Now we select {b,c} edge based on order that doesn't produce cycle.

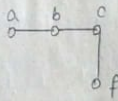


c is next working node.

The adjacent edges of vertex 'c'

{c,b} {c,f}

Now we select {c,f} edge based on order that doesn't produce cycle.

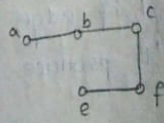


f is next working node.

The adjacent edges of vertex 'f'

{f,c} {f,e} {f,g} {f,j}

Now we select {f,e} edge based on order that doesn't produce cycle.



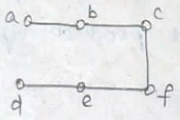
Scanned by CamScanner

e is next working node.

The adjacent edges of vertex 'e'

{e,b} {e,d} {e,f} {e,i}

Now we select {e,d} edge based on order that doesn't produce cycle.

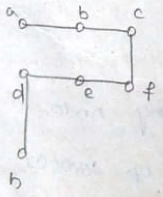


d is next working node.

The adjacent edges of vertex 'd'

{d,a} {d,e} {d,h}

Now we select {d,h} edge based on order that doesn't produce cycle.

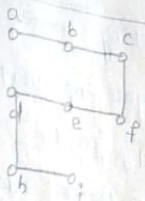


h is next working node.

The adjacent edges of vertex 'h'

{h,d} {h,i} {h,k}

Now we select {h,i} edge based on order that doesn't produce cycle.

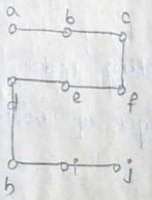


i is next working node.

The adjacent edges of vertex 'i'

{i,e} {i,h} {i,j} {i,k}

Now we select {i,j} edge based on order that doesn't produce cycle.

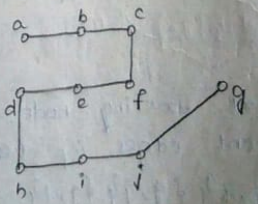


j is next working node.

The adjacent edges of vertex 'j'

{j,i} {j,f} {j,g}

Now we select {j,g} edge based on order that doesn't produce cycle.



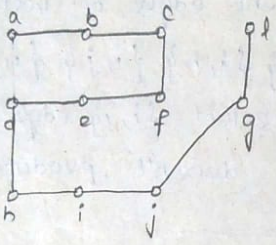
Scanned by CamScanner

g is next working node

The adjacent edges of vertex 'g'

{g,f} {g,j} {g,l}

Now we select {g,l} ~~vertex~~ edge based on edges that doesn't produce cycle.



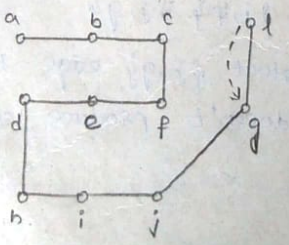
d is next working node.

The adjacent edges of vertex 'l'

{l,g}.

No more edge is selected.

Now move to parent node of 'l'



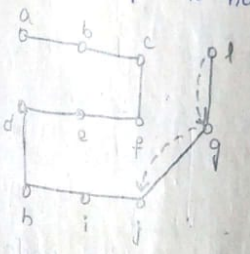
g is next working node

The adjacent edges of vertex 'g'

{g,f} {g,j} {g,l}

No more edge is selected

Now move to parent node of 'g'



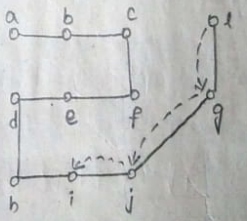
j is next working node

The adjacent edges of vertex 'j'

{j,f} {j,i} {j,g}

No more edge is selected.

Now move to parent node of 'j'



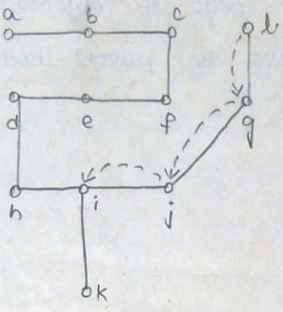
i is next working node.

The adjacent edges of vertex 'i'

{i,e} {i,h} {i,j} {i,k}

Now we select {i,k} edge based on edges that doesn't produce cycle.

Scanned by CamScanner

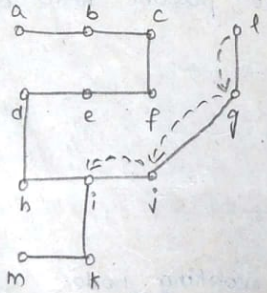


k is next working node

The adjacent edges of vertex 'k'

{k,h} {k,i} {k,m}

Now we select {k,m} edge based on order that doesn't produce cycle.



Result order:

a b c f e d h i j g l k m

* Breadth First Search (BFS) Algorithm:

BFS is used to find the spanning tree (T) of given graph (G).

Input: A connected graph G with n vertices $(V_1, V_2, \ldots V_n)$.

Output: Spanning tree.

Algorithm:

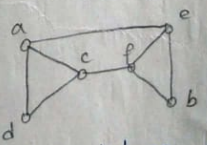
Step-1: let V_i be the root node of T.

Step-2: Adding all adjacent edges of vertex V_i, that are doesn't produce any cycle.

Step-3: Repeat step-2 until all vertices are visited. After all vertices are visited then stop. Then T is the spanning tree of given graph.

Example:

1. Implement spanning tree of following graph using BFS.



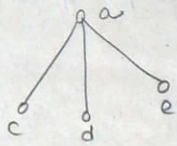
Sol. We select 'a' as root of spanning tree T.

Scanned by CamScanner

Adjacent edges of 'a' are

{a,c} {a,d} {a,e}

then we add all edges to vertex 'a'

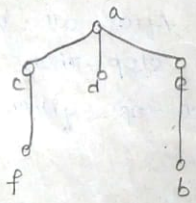


Next, we need to find out adjacent edges of all vertices in level 1

adjacent edges are {c,a} {c,d} {c,f}

{d,a} {d,c} {e,a} {e,b} {e,f}

add all adjacent edges, that doesn't produce cycle.



This is spanning tree of given graph.

Result Order:

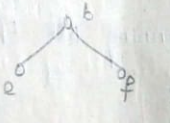
a c d e f b

We select 'b' as root of spanning tree T

Adjacent edges of 'b' are

{b,e} {b,f}

then we add all edges to vertex 'b'

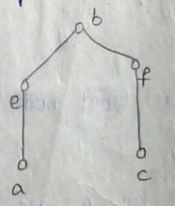


Next, we need to find out adjacent edges of all vertices in level 1

adjacent edges are {e,a} {e,f} {e,b}

{f,e} {f,c} {f,b}

add all adjacent edges, that doesn't produce cycle.



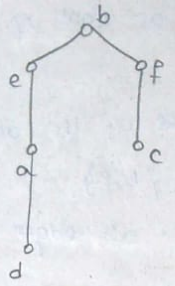
Next, we need to find out adjacent edges of all vertices in level 2

adjacent edges are {a,c} {a,d} {a,e}

{c,a} {c,d} {c,f}

add all adjacent edges, that doesn't produce cycle.

Scanned by CamScanner

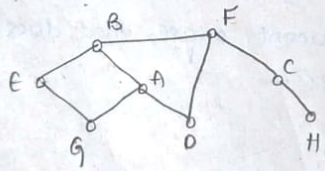


This is spanning tree of given graph.

Result Order:

b e f a c d

2. Implement spanning tree of following graph using BFS.



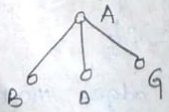
Sol. We select 'A' as root node of spanning tree.



Adjacent edges of A are

{A, B} {A, D} {A, G}

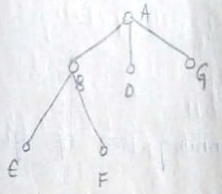
then we add all edges to vertex 'A'



Next, we need to find out adjacent edges of all vertices in level 1.

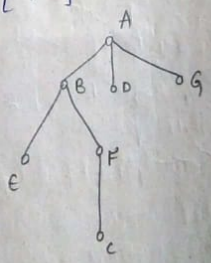
adjacent edges are {B, A} {B, E} {B, F}. {D, A} {D, F} {G, A} {G, E}

add all adjacent edges that doesn't produce cycle.



Next, we need to find out adjacent edges of all vertices in level 2

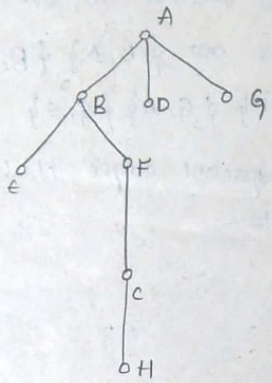
adjacent edges are {E, B} {E, G} {F, B} {F, C} {F, D}



Next, we need to find out adjacent edges of all vertices in level 3.

Scanned by CamScanner

adjacent edges are {C, F} {C, H}

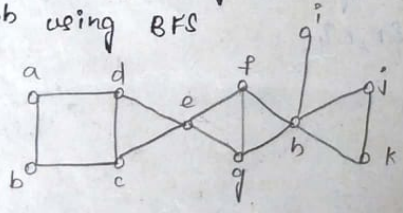


This is spanning tree of given graph.

Result Order:

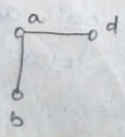
A B D G E F C H

3. Implement spanning tree of following graph using BFS

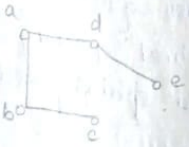


Sol:

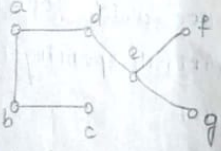
{a, b} {a, d}



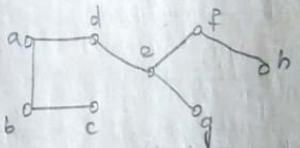
{d, a} {d, e} {d, c} {b, a} {b, c}



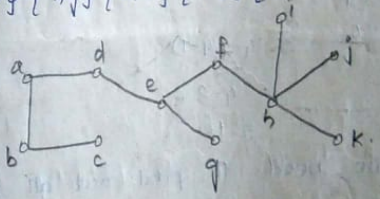
{c, b} {c, e} {c, d} {e, d} {e, c} {e, f} {e, g}



{f, e} {f, h} {g, e} {g, h} {f, g}



{h, f} {h, g} {h, i} {h, j} {h, k}



This is the spanning tree of graph.

Result Order:

a b d c e f g h i j k.

Scanned by CamScanner

09/10/14

* Minimal Spanning Tree (MST)

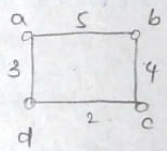
Let us consider graph G is a weighted graph. A minimal spanning tree of graph G is a spanning tree of G with minimum weight.

(or)

A spanning tree which has least weight is called a minimal spanning tree (MST).

Eg:

1. Find the MST of given graph.



Sol. Circuit rank = m-(n-1)

m = 4

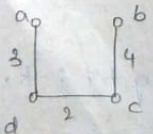
n = 4 = 4-(4-1)

= 4-3

= 1.

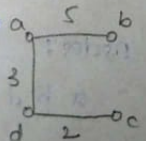
Now, we need to find out all possible spanning trees.

(i)



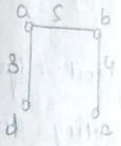
Weight = 3+2+4 = 9

(ii)



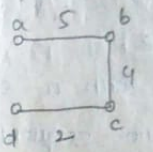
weight = 5+3+2 = 10.

(iii)



Weight = 5+3+4 = 12

(iv)



weight = 5+4+2 = 11

The spanning tree (i) have least weight so, it is minimal spanning tree of given graph.

We have one algorithm to obtain minimal spanning tree of given graph. i.e, Kruskal's Algorithm.

Kruskal's Algorithm:

It is used to obtain minimal spanning tree (MST).

Input:

A connected weighted graph G.

Output:

Minimal spanning tree of graph G.

Algorithm:

Step-1:

Given weighted graph with n vertices, list the all edges of G in order of

Scanned by CamScanner

non-decreasing weights.

Step-2:

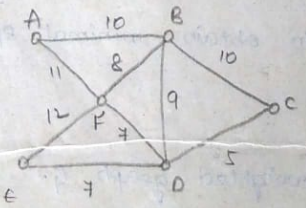
Start with a smallest weighted edge and continue sequentially by selecting one edge at a time, so that it doesn't form a cycle.

Step-3:

When n-1 edges are selected, stop the step-2 process. Then it is a MST.

Example:

1. Use Kruskal's Algorithm to find MST of given graph.

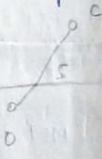


Sol. The above graph have 6 vertices, then we need to obtain spanning tree with (6-1)5 edges.

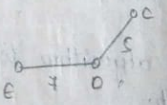
Now, write all edges in non decreasing order based on weights.

Edge	CD	DE	DF	FB	BD	BC	AB	AF	FE
Weight	5	4	7	8	9	10	10	11	12

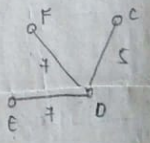
Now, we select edge CD.



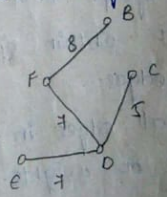
Next we select edge DE



Next we select edge DF

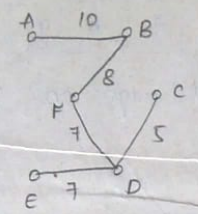


Next we select edge FB



Scanned by CamScanner

Next, BD and BC edges forms cycles so we select AB edge

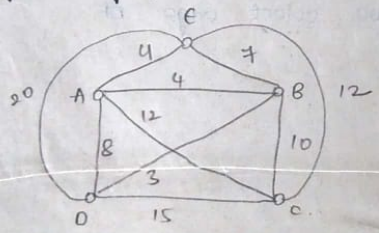


This is the MST

Weight of MST is

$5+7+7+8+10 = 37$.

2. Use kruskal's algorithm to find MST of given graph.

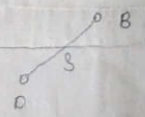


Sol. The above graph have 5 vertices, then we need to obtain spanning tree with (5-1) 4 edges.

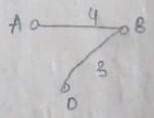
Now, write all edges in non decreasing order based on weights.

Edge	BD	AB	AE	BE	AD	BC	CE	AC	CD	DE
Weight	3	4	4	7	8	10	12	12	15	20

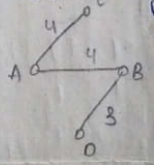
Now we select edge BD



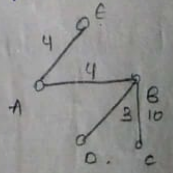
Next we select edge AB



Next we select edge AE



Next we select edge BE, BE and AD edges forms cycles, so we select BC edge.



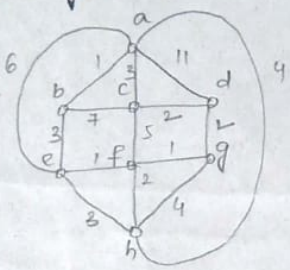
This is the MST.

Weight of MST is $3+4+4+10 = 21$.

Scanned by CamScanner

10/10/14.

1. Find MST of the following graph using Kruskal's Algorithm.



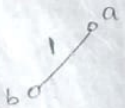
Sol. The above graph have 8 vertices, then we need to obtain spanning tree with (8-1)7 edges.

Now, write all edges in non decreasing order based on weights.

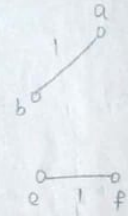
Edge	~~ab~~ ab	ef	fg	fh	cd	dg	be
Weight	1	1	1	2	2	2	3

eh	ac	ah	hg	cf	ae	bc	ad
3	3	4	4	5	6	7	11

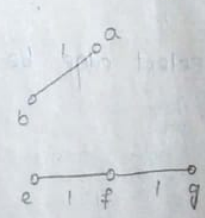
Now we select edge ab



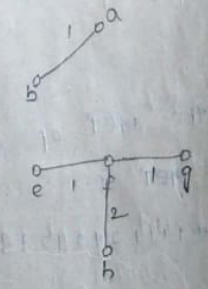
Next we select edge ef



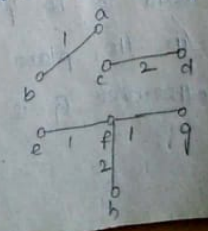
Next we select edge fg.



Next we select edge fh

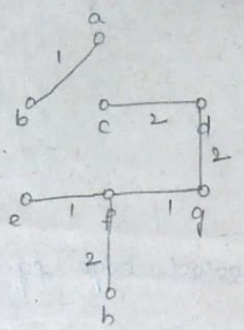


Next we select edge cd.

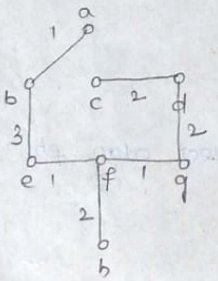


Scanned by CamScanner

Next we select edge dg.



Next we select edge be



This is the MST of graph.

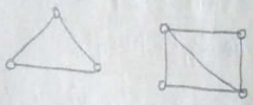
Weight of MST is

$1+1+1+2+2+2+3 = 12$

Planar Graph:

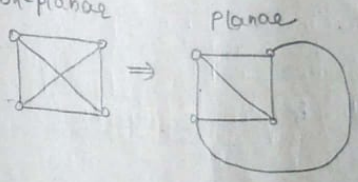
A graph G is said to be planar if it can be drawn in the plane without its edges crossing. otherwise G is non-planar.

Eg:



Eg:

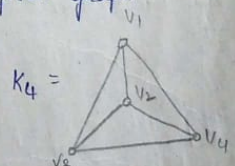
Non-planar ⇒ Planar



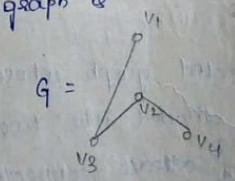
Sub Graph:

A subgraph is a graph which is a part of another graph.

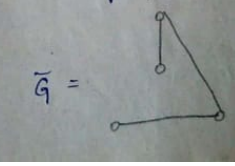
Eg: Given graph.



Subgraph is



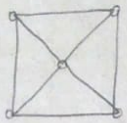
Complement graph is $\bar{G} = K - G$



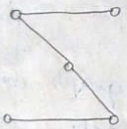
Scanned by CamScanner

Eg:

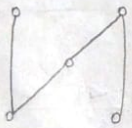
Given graph is



Subgraph is



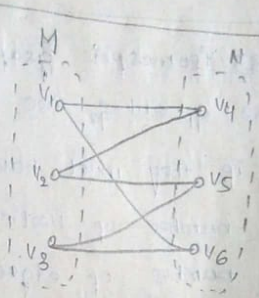
Complement graph is



Bipartite Graph:

It is an undirected graph whose set of vertices can be divided into two sets. and each vertex is atleast connected with two vertices

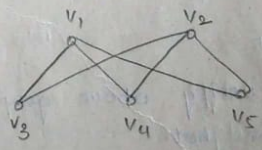
Eg:



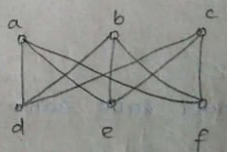
Complete Bipartite graph:

It is a bipartite graph in which every vertex of 'm' is adjacent to every every vertex of 'n' and it is denoted as $k_{m,n}$ where m,n are set of vertices

Eg: Draw $k_{2,3}$



Draw $k_{3,3}$.



Draw $k_{1,5}$



Scanned by CamScanner

Module-4

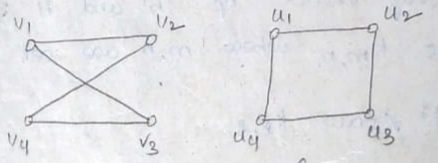
* Isomorphism (or) Isomorphic graphs:

Two graphs G_1 and G_2 are isomorphic if and only if they must have.

1. The same number of vertices.
2. The same number of edges.
3. Same degree sequences.

Example:

1. Show that the following graphs are Isomorphic.



G_1 G_2

Sol: In order to verify above graphs are isomorphic or not.

The following conditions are to be satisfied.

Condition-1:

Both graphs must have same number of vertices.

$G_1 = 4$ vertices

$G_2 = 4$ vertices.

$\therefore$ condition 1 is satisfied.

Condition-2:

Both graphs must have same number of edges.

$G_1 = 4$ edges

$G_2 = 4$ edges.

$\therefore$ Condition 2 is satisfied.

condition-3:

Both graphs must have same degree sequence.

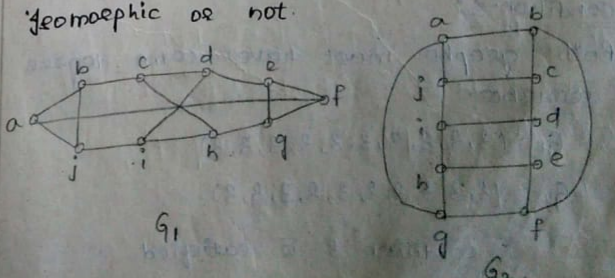
$G_1 = (2, 2, 2, 2)$

$G_2 = (2, 2, 2, 2)$

$\therefore$ condition 3 is satisfied.

$\therefore$ Both graphs are Isomorphic.

2. Show (that whether the) following graphs are Isomorphic or not.



G_1 G_2

Scanned by CamScanner

Sol. Inorder to verify above graphs are isomorphic or not.

The following conditions are to be satisfied.

condition-1:

Both graphs must have same number of vertices.

G_1 = 10 vertices

G_2 = 10 vertices.

∴ condition 1 is satisfied.

condition-2:

Both graphs must have same number of edges

G_1 = 15 edges.

G_2 = 15 edges

∴ condition 2 is satisfied.

condition-3:

Both graphs must have same degree sequence.

G_1 = {3, 3, 3, 3, 3, 3, 3, 3, 3, 3}

G_2 = {3, 3, 3, 3, 3, 3, 3, 3, 3, 3}

∴ condition 3 is satisfied.

∴ Both graphs are Isomorphic.

* Euler Graph:

A connected graph G is a Euler graph if and only if it contains a Euler circuit.

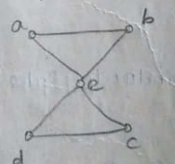
Defination:

Euler Circuit:

A Euler circuit or line is a path which includes each edge of given graph exactly once and intersects each vertex atleast once.

Example:

1. Find the given graph is Euler graph or not.



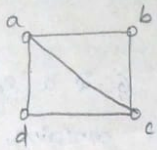
Sol. We need to find out Euler circuit of given graph.

Euler circuit is a-b-e-c-d-e-a

The above graph has the euler circuit so the graph is Euler Graph.

Scanned by CamScanner

2. Find the given graph is Euler graph or not.

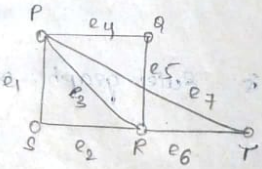


Sol. We need to findout Euler circuit of given graph.

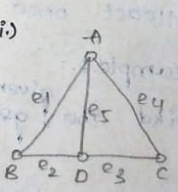
The above graph has no Euler circuit so the graph is not Euler graph.

3. Find the given graphs are Euler graphs or not.

(i)



(ii)



Sol. (i) We need to findout Euler circuit of given graph.

Euler circuit is P-S-R-P-Q-R-T-P

The above graph has the Euler circuit so the graph is Euler graph.

(ii) We need to findout Euler circuit of given graph.

The above graph has no Euler circuit.

So the graph is not Euler graph.

Scanned by CamScanner